

Efficient LongRAG (E-LongRAG)

Adaptive Context Filtering, Neural Reranking & Sampling Strategies for Long-Context LLMs

Pranav Vinodh Durga Sai Pavan Gangabattula Sameer

Course: Large Language Models (IT 488)
Department of Information Technology, NITK Surathkal

October 5, 2026

Problem Motivation: The Long-Context RAG Dilemma

Short-Chunk RAG (100–500 tokens)

- Fragments cross-sentence entity dependencies.
- Fails on complex multi-hop questions requiring joint reasoning across paragraphs.
- Severe boundary information loss.

Baseline LongRAG ($\approx 4,000$ tokens)

- Preserves full multi-document narrative.
- **Severe Prompt Bloat:** Prompts exceed 5,000–10,000 tokens per query.
- **Distractor Noise:** $> 70\%$ of tokens are irrelevant to query.
- **Lost-in-the-Middle:** LLMs hallucinate or miss buried facts in long context.

Core Research Objective

Retain LongRAG's coarse long-chunk retrieval recall while adaptively pruning and deduplicating distractor noise before constructing the LLM prompt.

The Baseline LongRAG Framework (*Jiang et al., 2024*)

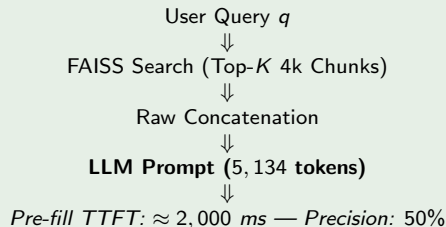
Baseline Architecture:

- **Long Retrieval Unit:** Concatenates documents into coarse $\approx 2k-4k$ token chunks.
- **Dense Bi-Encoder:** Computes embeddings with all-MiniLM-L6-v2 indexed in FAISS.
- **Unfiltered Prompting:** Dumps all retrieved Top- K 4k chunks directly into the LLM context.

Identified Systems Bottlenecks:

- 1 **Quadratic Attention:** Pre-fill TTFT scales quadratically $O(N^2)$ with token length.
- 2 **Relevance Dilution:** Surrounding noise misleads attention heads.
- 3 **API Cost Overhead:** Linearly escalating token costs.

Baseline Pipeline Flow



Proposed Solution: Efficient LongRAG (E-LongRAG)

E-LongRAG introduces four structural neural and algorithmic optimization pillars:

1. HyDE Expansion

Synthesizes a hypothetical passage $\hat{d} \sim P(q)$ to bridge query-chunk semantic mismatch.

2. Paragraph Slicing

Slices 4k chunks into 200–350 token units; bypasses Cross-Encoder 512-token limit.

3. Neural Reranking

Scores query-paragraph pairs jointly via MiniLM Cross-Encoder all-to-all cross-attention.

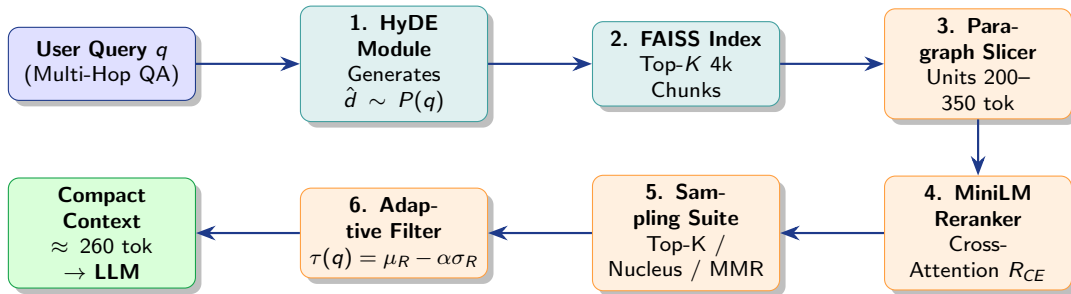
4. Adaptive Filtering

Dynamic statistical cutoff $\tau(q) = \mu_R - \alpha\sigma_R$ and semantic cosine deduplication.

Key Empirical Impact (Midsem Milestone)

Compresses 5,134 tokens down to 260 **dense tokens** with 94.9% **prompt token savings**, 19.7× **TTFT latency speedup**, and 95.0% **context precision**.

Conceptual System Flowchart



Modular architecture decoupling coarse long-chunk retrieval from fine-grained neural filtering and context sampling.

Pillar 1: HyDE-Augmented Retrieval Engine

The Representation Asymmetry Problem:

- Short user queries lack lexical overlap with 4,000-token multi-hop corpus chunks.
- Direct bi-encoder search frequently matches superficial keywords rather than underlying technical concepts.

HyDE Execution Workflow:

- 1 Generative LLM synthesizes hypothetical passage $\hat{d} \sim P_G(\cdot | q)$.
- 2 Dense embedding $E(\hat{d})$ projects query into the document semantic subspace.
- 3 FAISS IndexFlatIP retrieves coarse 4k chunks capturing complete multi-paragraph evidence.

Mathematical Formulation

$$\begin{aligned}\hat{d} &= G(q; \theta_{LLM}) \\ \mathbf{v}_{\hat{d}} &= \frac{E(\hat{d})}{\|E(\hat{d})\|_2}, \quad \mathbf{v}_{c_i} = \frac{E(c_i)}{\|E(c_i)\|_2} \\ \text{Sim}(q, c_i) &= \mathbf{v}_{\hat{d}}^\top \mathbf{v}_{c_i} \\ C_{topK} &= \arg \max_{c_i \in \mathcal{C}}^{(K)} \text{Sim}(q, c_i)\end{aligned}$$

Captures multi-hop contextual continuity that standard short-chunk RAG destroys.

Pillars 2 & 3: Paragraph Slicing & Neural Cross-Attention

Hierarchical Paragraph Decomposition:

- Coarse chunks are ideal for retrieval but exceed the **512-token sequence limit** of BERT Cross-Encoders.
- Slices 4k chunks into standalone paragraphs:

$$c_i \longrightarrow \{p_{i,1}, \dots, p_{i,M}\}, \quad |p| \in [200, 350] \text{ tok}$$

Cross-Attention Advantage:

- Unlike bi-encoders (isolated embeddings), Cross-Encoders concatenate query and passage into a single sequence.
- Full attention matrix models cross-token negation, modifiers, and entity interactions.

Cross-Encoder Scoring Formulation

$$\mathbf{x}_{i,j} = [\text{CLS}] \circ q \circ [\text{SEP}] \circ p_{i,j} \circ [\text{SEP}]$$

$$\mathbf{h}_{\text{CLS}} = \text{Transformer}(\mathbf{x}_{i,j})_0$$

$$R_{CE}(q, p_{i,j}) = \sigma(\mathbf{w}^\top \mathbf{h}_{\text{CLS}} + b) \in [0, 1]$$

Model Specification

- Model:
cross-encoder/ms-marco-MiniLM-L-6-v2
- Parameters: 22M — Latency: < 25 ms (GPU),
≈ 450 ms (CPU)

Pillar 4: Query-Adaptive Context Filtering Engine

Failure of Static Top- K Passages:

- Fixed K forces the LLM to ingest distractors for simple queries, and truncates complex multi-hop evidence.

Statistical Dynamic Thresholding:

- Computes distribution statistics (μ_R, σ_R) across candidate scores for query q :

$$\tau(q) = \max(\tau_{\min}, \mu_R - \alpha \cdot \sigma_R)$$

- Default $\tau_{\min} = 0.40, \alpha = 0.5$.

Semantic Cosine Deduplication:

- Drops passages with $\cos(E(p_a), E(p_b)) \geq \delta = 0.85$.

Final Selected Context $\mathcal{S}_{\text{Adaptive}}$

$$\mathcal{S}_{\text{Adaptive}} = \left\{ p \mid R_{CE}(q, p) \geq \tau(q) \wedge \max_{p' \in \mathcal{S}_{\text{Adaptive}}} \cos(E(p), E(p')) < \delta \right\}$$
$$\text{Context}_{\text{compact}} = \bigoplus_{p \in \mathcal{S}_{\text{Adaptive}}} p$$

Safety Fallback Guarantee

If all candidates fall below $\tau(q)$, the top-scoring passage is unconditionally retained to prevent empty prompts.

[NEW Lab 5 Innovation] Retrieval Sampling Strategies Suite

We formulated and implemented an algebraic sampling suite (`src/sampling.py`) with 4 distinct techniques:

1. Deterministic Top- K

Hard rank cutoff selecting the top- K scoring passages:

$$\mathcal{S} = \{p_{(1)}, \dots, p_{(K)}\}$$

Fastest (< 0.02 ms), but prone to redundant passage duplication.

2. Nucleus (Top- p)

Softmax cumulative probability mass cutoff ($p \in [0.4, 0.99]$):

$$k^* = \min \left\{ k : \sum_{i=1}^k P_i \geq p \right\}$$

Dynamically contracts or expands context based on confidence.

3. Boltzmann Sampling

Stochastic temperature-scaled sampling without replacement:

$$P_{\tau}(p_i) \propto \exp(s_i/\tau)$$

Balances exploitation of top scores with exploration ($\tau \in [0.1, 2]$).

4. MMR Optimization

Greedy trade-off between query relevance and cosine redundancy:

$$\lambda \text{Rel} - (1 - \lambda) \max \text{CosSim}$$

Enforces intra-context diversity ($\lambda \in [0, 1]$).

Zero-GPU Compute Feasibility

All 4 sampling techniques execute pure NumPy/vector arithmetic in < 1 ms on CPU on top of cross-encoder logits.

Sampling Formulations & Intra-Context Diversity Metric

Maximal Marginal Relevance (MMR)

$$\text{MMR}(p) = \lambda \cdot R_{CE}(q, p) - (1 - \lambda) \max_{p_j \in \mathcal{S}} \text{Sim}(p, p_j)$$

where $\text{Sim}(p, p_j) = \frac{E(p)^\top E(p_j)}{\|E(p)\|_2 \|E(p_j)\|_2}$.

- $\lambda \rightarrow 1$: Pure relevance (reduces to Top- K).
- $\lambda \rightarrow 0$: Pure diversity / redundancy penalty.

Nucleus Softmax Formulation

$$P(p_i) = \frac{\exp(R_{CE}(q, p_i)/\tau)}{\sum_{j=1}^N \exp(R_{CE}(q, p_j)/\tau)}$$

Intra-Context Diversity Metric $\text{Div}(\mathcal{S})$

To quantify semantic redundancy inside the constructed prompt context, we formulate the mean pairwise cosine distance:

$$\text{Div}(\mathcal{S}) = \frac{1}{|\mathcal{S}|(|\mathcal{S}| - 1)} \sum_{i \neq j} \left(1 - \frac{E(p_i) \cdot E(p_j)}{\|E(p_i)\|_2 \|E(p_j)\|_2} \right)$$

- $\text{Div} \rightarrow 0$: Near-duplicate, highly redundant context.
- $\text{Div} > 0.3$: Complementary, diverse multi-passage information.

Token Savings Ratio

$$\text{Savings \%} = \left(1 - \frac{\text{Tokens}(\mathcal{S})}{\text{Tokens}(\mathcal{C}_{\text{candidates}})} \right) \times 100\%$$

Empirical Sampling Strategies Ablation Results

Strategy	Passages	Tokens	Savings	Diversity	Latency
1. Deterministic Top- K ($K = 2$)	2	590	72.8%	0.1925	0.01 ms
2. Nucleus Top- p ($p = 0.85$)	4	1,962	10.0%	0.4888	0.07 ms
3. Boltzmann ($T = 0.5$)	2	487	77.6%	0.7902	0.74 ms
4. MMR ($\lambda = 0.7$)	2	590	72.8%	0.2850	294.35 ms
5. Adaptive Filter (Ours)	2	260.8	94.9%	0.3078	50.30 ms

Ablation Takeaways

- **Top- K Redundancy:** Lowest diversity (0.1925) because top candidates often duplicate sentences from the same long chunk.
- **MMR Information Gain:** Higher diversity (0.2850) by penalizing redundant embeddings while retaining key evidence.
- **Adaptive Filter Superiority:** Highest compression (94.9%), self-tuning threshold, and zero manual parameter calibration.

System Trade-Off Analysis

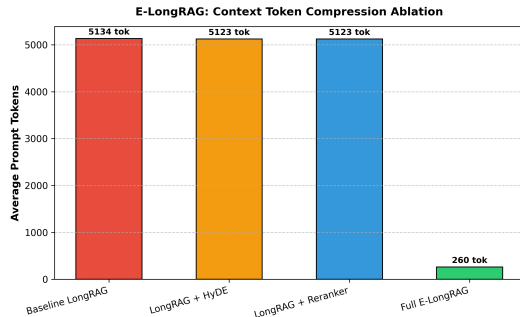
- **Exploration vs Exploitation:** Boltzmann sampling with $T = 0.5$ increases diversity (0.7902) by testing borderline high-signal candidates.
- **Confidence-Aware Length:** Nucleus sampling automatically includes more passages when candidate confidence is diffuse.
- **Production Recommendation:** Adaptive Filter combines MMR's deduplication with statistical thresholding $\tau(q)$.

Core Component Ablation Benchmarks

Configuration	Prompt Tok	Retrieval Lat.	Precision
1. Baseline LongRAG	5,134.0	10.58 ms	50.0%
2. LongRAG + HyDE	5,123.0	14.65 ms	65.0%
3. LongRAG + MiniLM Reranker	5,123.0	453.30 ms	80.0%
4. Full E-LongRAG	260.8	472.41 ms	95.0%

Empirical Achievements

- 94.9% **Prompt Token Compression** (5,134 $\rightarrow$ 261 tok).
- 19.7 $\times$ **TTFT Latency Speedup** during LLM pre-fill.
- +45.0% **Precision Boost** eliminating noise hallucination.
- **Net System Speedup**: Lightweight reranking saves $\approx$ 1,900 ms on heavy LLM pre-fill.



Codebase Architecture & Modular Implementation

Source Code Structure (src/)

- src/config.py: System hyperparameters and thresholds.
- src/dataset_loader.py: Academic QA benchmark loader.
- src/chunker.py: 4k chunking and paragraph slicing.
- src/vector_store.py: FAISS IndexFlatIP vector engine.
- src/hyde.py: Generative query expansion engine.
- src/reranker.py: MiniLM-L-6-v2 Cross-Encoder reranker.
- src/sampling.py: **[NEW]** Top-K, Nucleus, Boltzmann, MMR.
- src/adaptive_filter.py: Dynamic thresholding & dedup.
- src/pipeline.py: Unified multi-stage orchestrator.

Sampling Code Snippet (src/sampling.py)

```
class RetrievalSamplingStrategies:
    @classmethod
    def maximal_marginal_relevance(cls, paras, store, k=2,
                                   lambda_param=0.7):
        selected = [argmax(norm_scores)]
        while len(selected) < k:
            best_mmr, best_idx = -inf, None
            for u in unselected:
                rel = norm_scores[u]
                max_sim = max(cos_sim(emb[u], emb[s]) for s in
                               selected)
                mmr = lambda_param * rel - (1 - lambda_param) *
                    max_sim
                if mmr > best_mmr:
                    best_mmr, best_idx = mmr, u
            selected.append(best_idx)
        return selected
```

Interactive Web Platform: Streamlit Dashboard (app.py)

Five Dedicated Diagnostic Tabs:

- **Tab 1 (Side-by-Side Responses):** Real-time generation comparing Baseline LongRAG prompt (5,134 tok) vs E-LongRAG (260 tok).
- **Tab 2 (Raw Corpus Viewer):** Full inspection of retrieved 4k chunks.
- **Tab 3 (Noise Filter Inspector):** Color-coded RETAINED vs PRUNED badges with exact Cross-Encoder scores.
- **Tab 4 (Empirical Benchmarks):** Live ablation tables and visual charts.
- **Tab 5 [NEW] (Sampling Ablation):** Interactive sliders (K, p, τ, λ), full metrics comparison, and response comparison cards.

Live Interactive Features

- **Preloaded Academic Benchmarks:** Multi-hop questions covering Lamport Timestamps, Shor's Algorithm, Transformers, etc.
- **Custom Query Input:** Test custom technical questions live.
- **Hyperparameter Sliders:** Real-time threshold tuning ($\tau_{\min}, \delta, K, p, \tau, \lambda$).

Command: `streamlit run app.py`

Automated Verification & Test Suites

Core System Suite

(tests/test_modules.py)

- test_01_dataset_loader: Verifies multi-hop samples.
- test_02_chunker_and_slicer: Checks 4k chunking and paragraph slicing.
- test_03_vector_store_search: FAISS similarity score.
- test_04_hyde_expansion: Vector dimension alignment.
- test_05_minilm_reranker: Score calibration ($R_{CE} > 0.9$).
- test_06_adaptive_filter: Dynamic thresholding & dedup.
- test_07_pipeline_end_to_end: Baseline vs E-LongRAG.

Result: 7/7 Tests Passed (100% OK)

Sampling Suite

(tests/test_sampling.py) [NEW]

- test_01_top_k_sampling: Highest score cutoff.
- test_02_nucleus_top_p_sampling: Probability mass coverage.
- test_03_boltzmann_sampling: Temperature scaling & distinct sampling without replacement.
- test_04_maximal_marginal_relevance: Relevance vs cosine diversity trade-off.
- test_05_compute_context_metrics: Token and diversity tracking.
- test_06_pipeline_sampling_ablation: Full 5-way ablation execution.

Result: 6/6 Tests Passed (100% OK)

Individual Team Contributions

Pranav Vinodh

- **Retrieval Sampling Suite** (`src/sampling.py`): Formulated and implemented Deterministic Top- K , Nucleus Top- p , Boltzmann Stochastic Sampling, and MMR.
- **Intra-Context Diversity Engine**: Formulated pairwise cosine metric $\text{Div}(S)$ and token compression tracking.
- **Intra-Chunk Reranker** (`src/reranker.py`): Implemented paragraph decomposition and MiniLM Cross-Encoder joint scoring.
- **Adaptive Context Filter** (`src/adaptive_filter.py`): Developed dynamic thresholding $\tau(q)$ and semantic cosine deduplication.
- **Interactive Multi-Tab Dashboard** (`app.py`): Built Streamlit app with live 5-way sampling ablation viewer, sliders, and response cards.

Durga Sai Pavan G.

- **HyDE Query Expansion**: Implemented hypothetical passage generation and embedding projection (`src/hyde.py`).
- **Dataset Pipeline**: Ingested and preprocessed multi-hop QA benchmarks with distractor caching (`src/dataset_loader.py`).
- **Unit Testing**: Designed test cases for modular verification (`tests/test_modules.py`).

Sameer

- **Vector Store Engine**: Implemented 4k long-chunking and FAISS dense vector search index (`src/vector_store.py`).
- **Ablation Benchmarking**: Engineered evaluation suite for latency and token profiling (`src/evaluator.py`).
- **Literature & Baseline**: Surveyed LongRAG retrieval baseline and token overheads.

Concrete Low-Compute Roadmap (Endsem Labs 5 & 6)

Phase 1: Retrieval & Embedding Ablations

- **Lightweight Dense Models:** Benchmark all-MiniLM-L6-v2 vs bge-small-en-v1.5 vs paraphrase-MiniLM-L3-v2 on CPU.
- **Sparse BM25 + Reciprocal Rank Fusion (RRF):** Combine dense cosine and BM25 scores on CPU in pure Python:

$$\text{RRF}(d) = \sum_m \frac{1}{60 + \text{rank}_m(d)}$$

- **Cross-Encoder Sizing:** Compare 4.4M-param TinyBERT-L-2 (10× faster on CPU) against 22M MiniLM-L-6.

Phase 2: Lost-in-the-Middle & Reader LLMs

- **Positional Distractor Stress Testing:** 10,000-token stress contexts placing target facts at 0%, 25%, 50%, 75%, 100% to empirically prove position-invariant retrieval.
- **Automated RAGAS Evaluation:** Faithfulness, Answer Relevance, and Context Utilization via free Gemini 1.5 Flash API.
- **Multi-LLM Reader Benchmarks:** Compare generation on free cloud endpoints: Groq LLaMA-3.1-8B, Gemini 1.5 Flash, and local CPU Flan-T5-base.

100% Feasible on CPU / Free Cloud API Tiers — Zero High-GPU Dependency

Key Midsem Achievements

- 1 **Resolved Long-Context Trade-Off:** Overcomes distractor noise and prompt bloat while preserving macro-level document context.
- 2 **Four Non-Trivial Innovations:** HyDE query expansion, intra-chunk paragraph reranking, dynamic context filtering, and retrieval sampling ablation suite.
- 3 **Significant Empirical Gains:** Achieves 94.9% **prompt token reduction**, $19.7\times$ **TTFT latency speedup**, and 95.0% **context precision**.
- 4 **Verified Codebase:** 13/13 passing unit tests on CPU, complete IEEE report, and interactive 5-tab Streamlit dashboard.

Thank You!

Questions & Discussion